Python_Code.py

I ran this code before giving it to you. It works, all three algorithms give correct results, and it takes about 30 seconds to run.

python

```
"""
Design and Analysis of Algorithms - Assignment 1
Searching a student record database by registration number

Three strategies are compared:
  A. Linear search   (iterative, brute force, no preprocessing)
  B. Binary search   (recursive, decrease and conquer, needs a sorted array)
  C. Hash lookup     (transform and conquer, needs a hash table)

Every search returns (position, comparisons) so that the experiments can
count the basic operation from the theory (key comparisons), not just time.

Run:     python Python_Code.py
Needs:   Python 3.8+ and matplotlib   (pip install matplotlib)
Output:  tables in the console and the graph sheet Output.png
"""

import random
import time
import matplotlib
matplotlib.use("Agg")          # draw to a file, no window needed
import matplotlib.pyplot as plt

SEED = 42
SIZES = [1_000, 10_000, 100_000, 1_000_000]
```

```
NUM_PRESENT = 50              # queries for keys that exist
NUM_ABSENT = 50               # queries for keys that do not exist
LOAD_FACTOR = 0.75            # hash table load factor n / m
SEARCHES_PER_DAY = 50_000 * 24    # Task 12: 50,000 searches per hour
NAMES = ["Linear", "Binary", "Hash"]



# ---------------------------------------------------------------- algorithms
def linear_search(a, key):
    """Scan from the start. Basic operation: a[i] == key."""
    comps = 0
    for i, value in enumerate(a):
        comps += 1
        if value == key:
            return i, comps
    return -1, comps



def binary_search(a, low, high, key):
    """Recursive search on a sorted list. One three-way comparison per call."""
    if low > high:
        return -1, 0
    mid = (low + high) // 2
    if a[mid] == key:
        return mid, 1
    if key < a[mid]:
        idx, c = binary_search(a, low, mid - 1, key)
    else:
        idx, c = binary_search(a, mid + 1, high, key)
    return idx, c + 1
```

```
def merge_sort(a):
    """Preprocessing for binary search. Returns (sorted list, comparisons)."""
    comps = 0

    def sort(lst):
        nonlocal comps
        if len(lst) <= 1:
            return lst
        mid = len(lst) // 2
        left, right = sort(lst[:mid]), sort(lst[mid:])
        merged, i, j = [], 0, 0
        while i < len(left) and j < len(right):
            comps += 1
            if left[i] <= right[j]:
                merged.append(left[i]); i += 1
            else:
                merged.append(right[j]); j += 1
        merged.extend(left[i:])
        merged.extend(right[j:])
        return merged

    return sort(a), comps



def is_prime(x):
    if x < 2:
        return False
    d = 2
    while d * d <= x:
        if x % d == 0:
            return False
        d += 1
```

```
    return True


def next_prime(x):
    x = max(x, 2)
    while not is_prime(x):
        x += 1
    return x


def build_hash(a):
    """Chained hash table, h(key) = key mod m, m a prime near n / 0.75."""
    m = next_prime(int(len(a) / LOAD_FACTOR))
    table = [None] * m
    for idx, key in enumerate(a):
        h = key % m
        if table[h] is None:
            table[h] = [(key, idx)]
        else:
            table[h].append((key, idx))
    return table, m


def hash_search(table, m, key):
    """Go to the bucket, scan its chain. Basic operation: key comparison."""
    bucket = table[key % m]
    if bucket is None:
        return -1, 0
    comps = 0
    for k, idx in bucket:
        comps += 1
        if k == key:
```

```
            return idx, comps
    return -1, comps


# ---------------------------------------------------------------- testing
def run_edge_tests():
    """Small hand-made cases: empty, single, first, last, missing, in-between."""
    assert linear_search([], 5)[0] == -1
    assert binary_search([], 0, -1, 5)[0] == -1
    assert linear_search([7], 7)[0] == 0
    assert binary_search([7], 0, 0, 7)[0] == 0
    a = [10, 20, 30, 40, 50]
    table, m = build_hash(a)
    for key, expected in [(10, 0), (30, 2), (50, 4), (5, -1), (60, -1), (35, -1)]:
        assert linear_search(a, key)[0] == expected
        assert binary_search(a, 0, len(a) - 1, key)[0] == expected
        assert hash_search(table, m, key)[0] == expected
    assert merge_sort([50, 20, 40, 10, 30])[0] == [10, 20, 30, 40, 50]
    print("Edge-case tests passed (empty, single, first, last, missing, between).")


def timed_run(search_fn, queries):
    """Run all queries, return raw results, avg microseconds, avg comparisons."""
    out = []
    start = time.perf_counter()
    for key in queries:
        out.append(search_fn(key))
    elapsed = time.perf_counter() - start
    avg_us = elapsed / len(queries) * 1e6
    avg_comps = sum(c for _, c in out) / len(queries)
    return out, avg_us, avg_comps
```

```
def verify(out, arr, queries, present, name):
    """Every answer must point at the right key, or be -1 for a missing key."""
    for (idx, _), key in zip(out, queries):
        if key in present:
            assert idx != -1 and arr[idx] == key, f"{name}: wrong result"
        else:
            assert idx == -1, f"{name}: found a key that is not there"



def run_size(n, rng):
    pool = rng.sample(range(10**9, 10**10), n + NUM_ABSENT)   # unique 10-digit numbers
    data, absent = pool[:n], pool[n:]
    present_keys = rng.sample(data, NUM_PRESENT)
    queries = present_keys + absent
    rng.shuffle(queries)
    present = set(present_keys)

    t = time.perf_counter()
    sorted_data, sort_comps = merge_sort(data)
    t_sort = time.perf_counter() - t
    t = time.perf_counter()
    table, m = build_hash(data)
    t_hash = time.perf_counter() - t

    runs = {
        "Linear": (lambda k: linear_search(data, k), data),
        "Binary": (lambda k: binary_search(sorted_data, 0, n - 1, k), sorted_data),
        "Hash": (lambda k: hash_search(table, m, k), data),
    }
    res = {"n": n, "sort_s": t_sort, "hash_s": t_hash, "sort_comps": sort_comps}
    for name, (fn, arr) in runs.items():
```

```
        out, avg_us, avg_c = timed_run(fn, queries)
        verify(out, arr, queries, present, name)
        res[name] = {"us": avg_us, "comps": avg_c}
    return res


def case_study(n=100_000):
    """Task 4: show best / worst behaviour on purpose-built keys."""
    rng = random.Random(SEED)
    data = rng.sample(range(10**9, 10**10), n)
    sdata, _ = merge_sort(data)
    table, m = build_hash(data)
    missing = 5                          # smaller than any 10-digit key
    print(f"\nBest / worst case check (n = {n:,})")
    print(f"  Linear  best  (first record)    : {linear_search(data, data[0])[1]:>8,} comparisons")
    print(f"  Linear  worst (last record)     : {linear_search(data, data[-1])[1]:>8,} comparisons")
    print(f"  Linear  worst (key missing)     : {linear_search(data, missing)[1]:>8,} comparisons")
    print(f"  Binary  best  (middle record)   : {binary_search(sdata, 0, n-1, sdata[(n-1)//2])[1]:>8,} comparisons")
    print(f"  Binary  worst (key missing)     : {binary_search(sdata, 0, n-1, missing)[1]:>8,} comparisons")
    sample = rng.sample(data, 1000)
    avg = sum(hash_search(table, m, k)[1] for k in sample) / len(sample)
    print(f"  Hash    typical (random keys)   : {avg:>8.2f} comparisons (average of 1,000)")
    # degenerate input: every key is a multiple of m, so all land in bucket 0
    small = 5_000
    m2 = next_prime(int(small / LOAD_FACTOR))
    bad = [i * m2 for i in range(1, small + 1)]
    t2, m2b = build_hash(bad)
    print(f"  Hash    worst (all keys collide) : {hash_search(t2, m2b, bad[-1])[1]:>8,} comparisons (n = {small:,})")


# ---------------------------------------------------------------------- output
```

```
def print_tables(results):
    print("\nAverage comparisons per search")
    print(f"{'n':>10}{'Linear':>12}{'Binary':>10}{'Hash':>8}")
    for r in results:
        print(f"{r['n']:>10,}{r['Linear']['comps']:>12,.1f}{r['Binary']['comps']:>10.1f}
{r['Hash']['comps']:>8.2f}")
    print("\nAverage time per search (microseconds)")
    print(f"{'n':>10}{'Linear':>12}{'Binary':>10}{'Hash':>8}")
    for r in results:
        print(f"{r['n']:>10,}{r['Linear']['us']:>12,.1f}{r['Binary']['us']:>10.2f}{r['Hash']['us']:>8.2f}")
    print("\nPreprocessing (seconds)")
    print(f"{'n':>10}{'Merge sort':>12}{'Sort comps':>14}{'Hash build':>12}")
    for r in results:
        print(f"{r['n']:>10,}{r['sort_s']:>12.3f}{r['sort_comps']:>14,}{r['hash_s']:>12.3f}")


def day4_costs(last):
    """Task 12: cost of one day (1.2 million searches) on the 1,000,000-record set."""
    prep = {"Linear": 0.0, "Binary": last["sort_s"], "Hash": last["hash_s"]}
    daily = {k: prep[k] + SEARCHES_PER_DAY * last[k]["us"] / 1e6 for k in NAMES}
    print(f"\nDay-4 scenario: {SEARCHES_PER_DAY:,} searches/day on {last['n']:,} records")
    print(f"{'Approach':>10}{'Prep (s)':>10}{'Searching (s)':>15}{'Total (s)':>12}")
    for k in NAMES:
        print(f"{k:>10}{prep[k]:>10.2f}{daily[k]-prep[k]:>15.2f}{daily[k]:>12.2f}")
    lin = last["Linear"]["us"] / 1e6
    for k in ("Binary", "Hash"):
        saving = lin - last[k]["us"] / 1e6
        print(f"Break-even vs linear search ({k}): about {prep[k] / saving:,.0f} searches")
    return daily


def draw_graphs(results, daily):
    ns = [r["n"] for r in results]
```

```
    fig, ax = plt.subplots(2, 2, figsize=(12, 9))
    style = {"Linear": ("tab:red", "o"), "Binary": ("tab:blue", "s"), "Hash": ("tab:green", "^")}

    for name in NAMES:
        c, mk = style[name]
        ax[0, 0].plot(ns, [r[name]["comps"] for r in results], color=c, marker=mk, label=name)
        ax[0, 1].plot(ns, [r[name]["us"] for r in results], color=c, marker=mk, label=name)
    ax[0, 0].set(title="Average comparisons per search", xlabel="n (records)",
ylabel="comparisons")
    ax[0, 1].set(title="Average time per search", xlabel="n (records)", ylabel="microseconds")
    for a in (ax[0, 0], ax[0, 1]):
        a.set_xscale("log"); a.set_yscale("log"); a.grid(True, which="both", alpha=0.3); a.legend()

    ax[1, 0].plot(ns, [r["sort_s"] for r in results], color="tab:blue", marker="s", label="Merge sort (for
binary)")
    ax[1, 0].plot(ns, [r["hash_s"] for r in results], color="tab:green", marker="^", label="Hash
build")
    ax[1, 0].set(title="Preprocessing time", xlabel="n (records)", ylabel="seconds")
    ax[1, 0].set_xscale("log"); ax[1, 0].set_yscale("log")
    ax[1, 0].grid(True, which="both", alpha=0.3); ax[1, 0].legend()

    bars = ax[1, 1].bar(NAMES, [daily[k] for k in NAMES], color=[style[k][0] for k in NAMES])
    ax[1, 1].set_yscale("log")
    ax[1, 1].set(title="Day-4: total cost of one day (1.2M searches, n = 1,000,000)",
ylabel="seconds (log scale)")
    for b, k in zip(bars, NAMES):
        ax[1, 1].text(b.get_x() + b.get_width() / 2, daily[k], f"{daily[k]:,.1f}s", ha="center", va="bottom")

    fig.suptitle("Searching a student record database: linear vs binary vs hash", fontsize=14)
    fig.tight_layout()
    fig.savefig("Output.png", dpi=150)
    print("\nGraphs saved to Output.png")
```

```
def main():
    run_edge_tests()
    rng = random.Random(SEED)
    results = []
    for n in SIZES:
        print(f"Running n = {n:,} ...", flush=True)
        results.append(run_size(n, rng))
    print("All three algorithms returned correct results on every query.")
    print_tables(results)
    case_study()
    daily = day4_costs(results[-1])
    draw_graphs(results, daily)


if __name__ == "__main__":
    main()
```

What I got when I ran it

Your timings will differ slightly, but the pattern will be the same.

n	Linear comparisons	Binary comparisons	Hash comparisons
1,000	748.9	9.4	1.07
10,000	7,652.5	12.8	1.17
100,000	77,820.8	16.3	1.13
1,000,000	729,212.5	19.5	0.99

At 1,000,000 records, a search took about 55,103 µs for linear, 7.9 µs for binary and 1.3 µs for hash. Preprocessing took 4.5 s for merge sort and 2.3 s for the hash build. For the Day-4 scenario, one day's total came to about 66,124 s for linear, 14.0 s for binary and 3.8 s for hash.

Where the results differ from Solution.pdf

Report these differences in your Report (they feed Task 10). None of them are errors.

Linear comparisons are about 0.75n, not 0.5n. The test queries are half present and half missing. Present keys average n/2 and missing keys cost n, so the mix averages 0.75n.

Hash comparisons are about 1.0 to 1.2, not 1.4. My 1.4 figure was for successful searches only. Missing keys that land in an empty bucket cost 0 comparisons, which pulls the average down.

Break-even points are higher than my estimates. Solution.pdf estimated about 40 searches for binary and about 2 for hash, because it counted operations. The measured times give about 82 and 42 searches. In Python, building the hash table (2.3 s) is relatively expensive compared with a single linear scan. This is a good example of theory and practice differing.

Hash is about 3.6 times cheaper than binary per day, not 16 times. The measured daily totals are 3.8 s against 14.0 s, which is smaller than the operation-count gap I gave. The conclusion holds: hashing still wins, but by a smaller margin than the theory suggested.

Merge sort made 18.7 million comparisons at 1M records, close to the n log n estimate of 20 million.